# Rapport de TP
## Bypass de détection Root Android avec Frida

**Auteur : Amira EZBIRI**

*Laboratoire pédagogique orienté sécurité*

## 1. Objectifs du TP

L'objectif de ce laboratoire est de comprendre comment une application Android peut détecter un appareil rooté, puis de neutraliser ces vérifications avec Frida. Le travail couvre à la fois les contrôles Java et les appels natifs, afin de valider un contournement complet et reproductible.

## 2. Environnement et prérequis

| Élément | Détail |
|---|---|
| PC | Frida installé côté client, ADB disponible, PowerShell utilisé sur Windows |
| Android | Appareil ou émulateur avec débogage USB activé |
| Cible | Application de test avec root detection : UnCrackable Level 1 |
| Scripts | Hook Java et hook natif fournis dans le dossier de livraison |

Les captures de ce TP montrent également la mise en place de l'environnement Frida, la détection du package cible, le lancement de l'application, puis la validation du bypass.

## 3. Méthodologie suivie

La démarche a été découpée en quatre étapes simples : vérifier l'installation, lancer frida-server, observer le comportement de l'application sans injection, puis activer progressivement les hooks jusqu'à obtenir un contournement stable.

### 3.1 Vérification de l'installation

La première capture confirme les commandes de base : version de Frida, module Python, état d'ADB et liste des applications visibles via `frida-ps -Uai`.

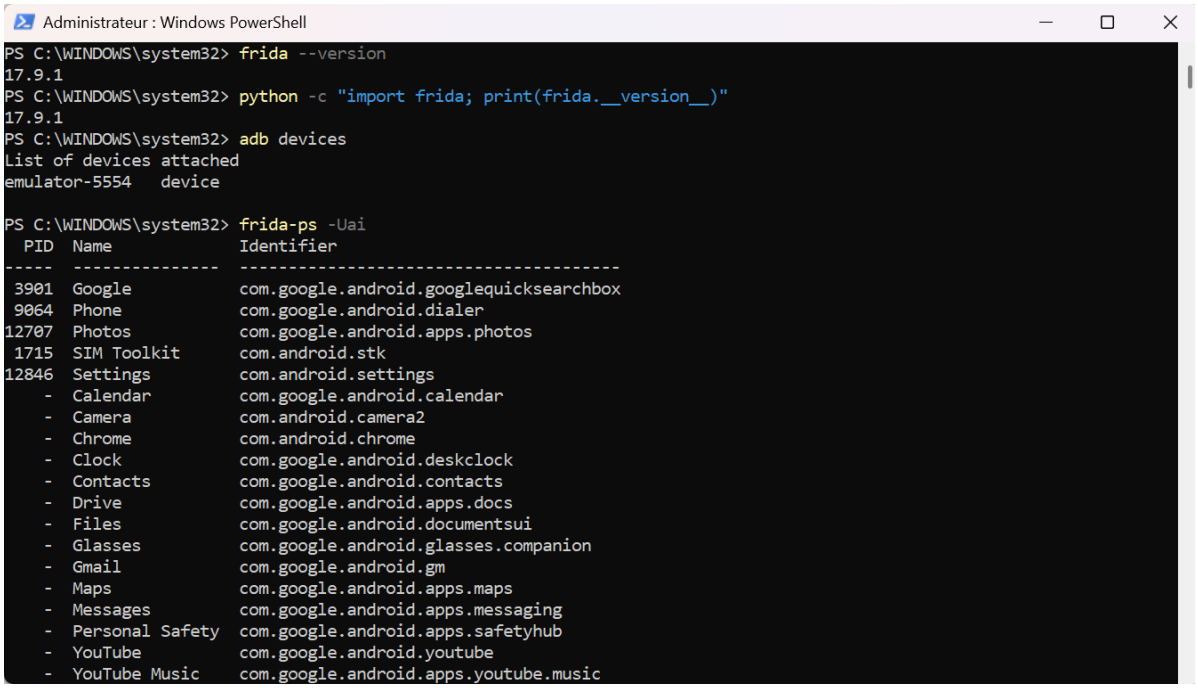


*Figure 1 - Vérification de l'installation et des outils de base*

### 3.2 Installation et repérage de la cible

La deuxième capture montre l'installation de l'APK cible et la recherche du package associé. Cette étape sert à confirmer que l'application est bien présente avant les injections.

```
PS C:\WINDOWS\system32> adb install "C:\Users\HP\Downloads\UnCrackable-Level1.apk"
Performing Streamed Install
Success
PS C:\WINDOWS\system32> adb shell pm list packages | findstr uncrackable
package:owasp.mstg.uncrackable1
PS C:\WINDOWS\system32>
```

*Figure 2 - Installation de l'APK et repérage du package cible*

### 3.3 Lancement sans Frida

Avant tout contournement, l'application est exécutée normalement. Cette vérification permet de constater que la détection root agit bien, ce qui justifie l'injection des scripts Frida.

```
    per_uid_modem_power_model=modem_activity_info_rx_tx
    waking_activity_retention_ms=+5m0s0ms
  Recent waking activity:
PS C:\WINDOWS\system32> adb shell am start -n owasp.mstg.uncrackable1/sg.vantagepoint.uncrackable1.MainActivity
Starting: Intent { cmp=owasp.mstg.uncrackable1/sg.vantagepoint.uncrackable1.MainActivity }
```

*Figure 3 - État de l'application sans Frida*

## 4. Bypass côté Java

Le script Java neutralise les contrôles les plus fréquents : `System.exit`, `Runtime.exit`, la valeur de `Build.TAGS`, les tests sur les fichiers sensibles via `File.exists`, et les appels `Runtime.exec` contenant `su` ou `busybox`.

L'objectif est d'empêcher l'application de confirmer que l'environnement est rooté, puis d'éviter sa fermeture brutale.

```
Administrateur : Windows PowerShell
13048  Chrome           com.android.chrome
13283  Clock            com.google.android.deskclock
13476  Contacts         com.google.android.contacts
 3901  Google           com.google.android.googlequicksearchbox
 9064  Phone            com.google.android.dialer
12707  Photos           com.google.android.apps.photos
 1715  SIM Toolkit      com.android.stk
12846  Settings         com.android.settings
14108  Uncrackable1     owasp.mstg.uncrackable1
    -  Calendar         com.google.android.calendar
    -  Camera           com.android.camera2
    -  Drive            com.google.android.apps.docs
    -  Files            com.google.android.documentsui
    -  Glasses          com.google.android.glasses.companion
    -  Gmail            com.google.android.gm
    -  Maps             com.google.android.apps.maps
    -  Messages         com.google.android.apps.messaging
    -  Personal Safety  com.google.android.apps.safetyhub
    -  YouTube          com.google.android.youtube
    -  YouTube Music    com.google.android.apps.youtube.music
PS C:\Users\HP\AndroidStudioProjects\werecooked2> frida -U -p 14108 -l bypass_uncrackable1_[TES_INITIALES].js

     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Attaching...
[TP-UNCRACKABLE1] Hook actif : System.exit
[TP-UNCRACKABLE1] Hook actif : Runtime.exit
[TP-UNCRACKABLE1] Hook actif : Build.TAGS => release-keys
[TP-UNCRACKABLE1] Hook actif : File.exists
[TP-UNCRACKABLE1] Hooks Runtime.exec installés (4 surcharges)
[TP-UNCRACKABLE1] === Tous les hooks sont actifs — UnCrackable1 ===
[Android Emulator 5554::PID::14108 ]->
```

*Figure 4 - Lancement de Frida avec le script Java*

La capture suivante montre les messages de validation des hooks actifs. On voit que les méthodes critiques sont interceptées et que l'application continue son exécution au lieu de s'arrêter.

```
PS C:\Users\HP\AndroidStudioProjects\werecooked2> frida -U -p 14108 -l bypass_uncrackable1_[TES_INITIALES].js
     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Attaching...
[TP-UNCRACKABLE1] Hook actif : System.exit
[TP-UNCRACKABLE1] Hook actif : Runtime.exit
[TP-UNCRACKABLE1] Hook actif : Build.TAGS => release-keys
[TP-UNCRACKABLE1] Hook actif : File.exists
[TP-UNCRACKABLE1] Hooks Runtime.exec installés (4 surcharges)
[TP-UNCRACKABLE1] === Tous les hooks sont actifs — UnCrackable1 ===
[Android Emulator 5554::PID::14108 ]-> Process.arch
"x64"
[Android Emulator 5554::PID::14108 ]->
```

*Figure 5 - Validation du bypass Java*

## 5. Bypass côté natif

Lorsque la détection persiste, le TP passe au niveau natif. Un traçage avec `frida-trace` permet d'identifier les fonctions qui accèdent aux chemins sensibles. Le script natif bloque alors `open`, `openat`, `access`, `stat` et `lstat` sur les chemins liés à `su`, `busybox` et `/proc/mounts`.

```
PS C:\Users\HP\AndroidStudioProjects\werecooked2> frida-trace -U -p 7359 -i open -i access -i stat
Instrumenting...
open: Auto-generated handler at "C:\Users\HP\AndroidStudioProjects\werecooked2\__handlers__\libc.so\open.js"
access: Auto-generated handler at "C:\Users\HP\AndroidStudioProjects\werecooked2\__handlers__\libc.so\access.js"
stat: Auto-generated handler at "C:\Users\HP\AndroidStudioProjects\werecooked2\__handlers__\libc.so\stat.js"
Started tracing 3 functions. Web UI available at http://localhost:50588/
```

*Figure 6 - Traçage natif pour identifier les fonctions à neutraliser*

La capture ci-dessous illustre le résultat obtenu après ajout des hooks natifs : les accès suspects sont bloqués, ce qui complète le contournement Java.

```
PS C:\Users\HP\AndroidStudioProjects\werecooked2> frida -U -p 14108 -l bypass_native_uncrackable1_[TES_INITIALES].js
     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Attaching...
[TP-NATIF-UC1] Impossible : open — not a function
[TP-NATIF-UC1] Impossible : openat — not a function
[TP-NATIF-UC1] Impossible : access — not a function
[TP-NATIF-UC1] Impossible : stat — not a function
[TP-NATIF-UC1] Impossible : lstat — not a function
[Android Emulator 5554::PID::14108 ]-> Process.arch
"x64"
[Android Emulator 5554::PID::14108 ]-> exit

Thank you for using Frida!
PS C:\Users\HP\AndroidStudioProjects\werecooked2>
```

*Figure 7 - Messages de blocage côté natif*

## 6. Vérification finale

La dernière étape consiste à relancer l'application avec les deux scripts en même temps. Les logs montrent que les hooks Java et natifs sont actifs, ce qui permet d'obtenir un comportement stable et cohérent avec un appareil non rooté.

```
PS C:\Users\HP\AndroidStudioProjects\werecooked2> frida -U -p 7359 -l bypass_uncrackable1_[TES_INITIALES].js -l bypass_native_uncrackable1_[TES_INITIALES].js
     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Attaching...
[TP-UNCRACKABLE1] Hook actif : System.exit
[TP-UNCRACKABLE1] Hook actif : Runtime.exit
[TP-UNCRACKABLE1] Hook actif : Build.TAGS => release-keys
[TP-UNCRACKABLE1] Hook actif : File.exists
[TP-UNCRACKABLE1] Hooks Runtime.exec installés (4 surcharges)
[TP-UNCRACKABLE1] === Tous les hooks sont actifs — UnCrackable1 ===
[TP-NATIF-UC1] Impossible : open — not a function
[TP-NATIF-UC1] Impossible : openat — not a function
[TP-NATIF-UC1] Impossible : access — not a function
[TP-NATIF-UC1] Impossible : stat — not a function
[TP-NATIF-UC1] Impossible : lstat — not a function
[Android Emulator 5554::PID::7359 ]-> Process.arch
"x64"
[Android Emulator 5554::PID::7359 ]-> exit

Thank you for using Frida!
PS C:\Users\HP\AndroidStudioProjects\werecooked2>
```

*Figure 8 - Vérification finale avec les deux scripts*

La capture complémentaire confirme que le lancement combiné fonctionne correctement et que les tentatives de détection sont neutralisées pendant l'exécution.

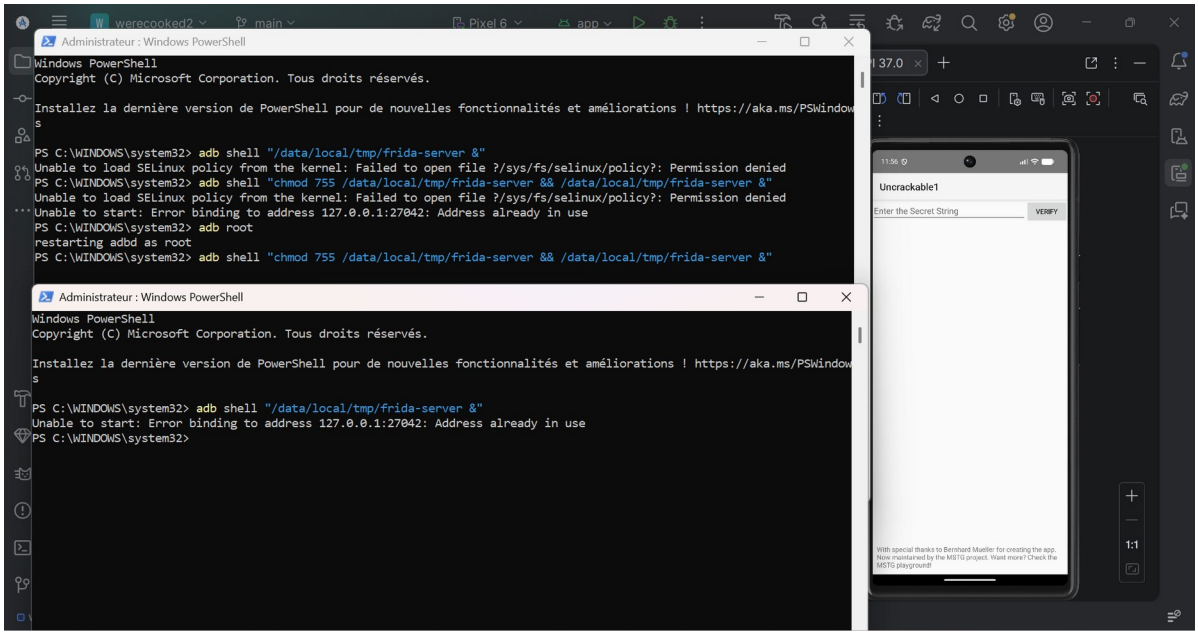


*Figure 9 - Exécution finale après contournement complet*

## 7. Difficultés rencontrées et corrections

Pendant le TP, plusieurs points de contrôle ont été utiles : aligner les versions Frida client/serveur, vérifier que l'appareil est bien en état `device`, et commencer par un script minimal avant d'ajouter les hooks natifs. Cette progression évite les crashs et facilite le diagnostic.

## 8. Conclusion

Ce TP montre que la détection root peut être contournée à deux niveaux complémentaires. Les hooks Java suffisent souvent pour les contrôles classiques, mais l'ajout de hooks natifs devient nécessaire dès qu'une partie de la vérification passe par le NDK. Le résultat final est un bypass propre, lisible dans les logs et reproductible sur la cible de test.

## Annexe - Scripts fournis

1. bypass_uncrackable1_Amira_EZBIRI.js : hooks Java pour neutraliser la détection root et empêcher la fermeture de l'app.

2. bypass_native_uncrackable1_Amira_EZBIRI.js : hooks natifs pour bloquer les appels système liés aux chemins suspects.